React.js: From Zero to Hero

A Comprehensive Guide & Personal Notes

Author: Ankit Kumar Singh
Date: September 09, 2025

Table of Contents

Part 1: The Foundation (Beginner)

1. Introduction: What is React? Why Learn It?

React is not a framework; it's a JavaScript library for building user interfaces, specifically for web applications. It was created by Facebook (now Meta) and is now maintained by a huge community.

Why React?

- **Component-Based:** You build your app from small, reusable pieces called "components." Think of them like LEGO blocks. A button is a component, a form is a component, a header is a component, and even a whole page is a component made of smaller components.
- **Declarative:** You describe what the UI should look like for any given state (e.g., "when the user is logged in, show the dashboard"). React takes care of updating the DOM to match that description. This is much simpler than the old imperative way of manually telling the browser how to update the DOM step-by-step.
- **Learn Once, Write Anywhere:** React can render on the web (React DOM), on mobile devices (React Native), and even on the server (Next.js, Gatsby).
- **Huge Ecosystem & Job Market:** It's incredibly popular, meaning tons of resources, libraries, and job opportunities.

Core Concept: React creates a virtual DOM (a lightweight copy of the real DOM) in memory. When a component's state changes, React updates the Virtual DOM first, compares it to a previous snapshot (a process called "diffing"), and then efficiently updates only the necessary parts of the real DOM. This is what makes React so fast.

2. Setting Up Your Environment

You don't need much to start, but here's the modern setup:

1. **Node.js and npm:** Download and install Node.js from nodejs.org (it includes npm, the Node Package Manager). This gives you access to a vast world of JavaScript tools and libraries.
2. **Create React App (CRA):** This is a comfortable, official toolchain for learning React. It sets up a modern build environment with everything configured (Babel, Webpack, ESLint, etc.) so you can focus on code.

```
npx create-react-app my-first-app
cd my-first-app
npm start
```

This will create a new directory my-first-app, install all dependencies, and start a development server at http://localhost:3000. Any saved changes will automatically reload the page!

3. Hello, World!: Your First React Component

Open the src/App.js file in your new project. You'll see something like this:

```
import logo from './logo.svg';
import './App.css';

function App() {
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Edit <code>src/App.js</code> and save to reload.
        </p>
        {/* ... more code ... */}
      </header>
    </div>
  );
}

export default App;
```

This App function is a functional component. It returns what looks like HTML, but it's actually **JSX**. Let's simplify it:

```
function App() {
  return (
    <div>
      <h1>Hello, World!</h1>
      <p>Welcome to my first React app.</p>
    </div>
  );
}

export default App;
```

Save the file. Your browser should instantly update to show "Hello, World!". Congratulations! You've just written your first component.

4. Understanding JSX: JavaScript XML

JSX is a syntax extension for JavaScript. It allows us to write HTML-like code inside our JavaScript.

Key Rules of JSX:

- Return a Single Element: A component must return a single parent element. This is why you often see things wrapped in a `<div>` or a special `<></>` (a React Fragment).

```
// ❌ Wrong: Two sibling elements
return (
  <h1>Title</h1>
  <p>Paragraph</p>
);

// ✅ Correct: Wrapped in a single parent
return (
  <div>
    <h1>Title</h1>
    <p>Paragraph</p>
  </div>
);

// ✅ Also Correct: Using a Fragment (doesn't add an extra node to the DOM)
return (
  <>
    <h1>Title</h1>
    <p>Paragraph</p>
  </>
);
```

- Use `className` instead of `class`: Since `class` is a reserved keyword in JavaScript, React uses `className`.

```
<div className="my-css-class">Content</div>
```

- **JavaScript inside JSX**: You can embed any JavaScript expression inside JSX using curly braces `{}`.

```
function Greeting() {
  const name = "Alice";
  const user = { firstName: 'Bob', lastName: 'Smith' };

  return (
    <div>
      <h1>Hello, {name}!</h1> // Output: Hello, Alice!
      <p>2 + 2 = {2 + 2}</p> // Output: 2 + 2 = 4
      <p>User: {user.firstName} {user.lastName}</p> // Output: User: Bob Smith
```

```
    </div>
  );
}
```

5. Components: The Building Blocks

Components are independent and reusable pieces of code. They come in two main types, but functional components are the modern standard.

1. **Functional Components:** JavaScript functions that return JSX.

```
// A simple Header component
function Header() {
  return <header><h1>My Website</h1></header>;
}

// To use it inside App:
function App() {
  return (
    <div>
      <Header /> {/* This is how you use a component */}
      <main>...</main>
    </div>
  );
}
```

2. **Class Components:** Older style using ES6 classes. You'll see these in older codebases, but for new projects, use functional components with Hooks.

```
import { Component } from 'react';

class Header extends Component {
  render() {
    return <header><h1>My Website</h1></header>;
  }
}
```

6. Props: Passing Data to Components

Props (short for properties) are how you pass data from a parent component down to a child component. They are read-only; the child cannot change its props.

```
// Child Component: Greeting
// It receives a `props` object as its argument.
```

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

// Parent Component: App
function App() {
  return (
    <div>
      {/* Pass a "name" prop to the Greeting component */}
      <Greeting name="Alice" />   // Output: Hello, Alice!
      <Greeting name="Bob" />     // Output: Hello, Bob!
      <Greeting name="Charlie" /> // Output: Hello, Charlie!
    </div>
  );
}
```

We often use destructuring to make the code cleaner:

```
// Destructuring the props object in the function parameter
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}
```

You can pass any data type as a prop: strings, numbers, arrays, objects, and even functions.

7. State: Adding Dynamic Behavior

If props are how data flows *down*, **state** is a component's own private data that can change over time. When state changes, the component re-renders to reflect those changes.

We manage state in functional components using the useState **Hook**.

What's a Hook? A Hook is a special function that lets you "hook into" React features (like state) from functional components. They always start with use.

```
// 1. Import the useState Hook
import { useState } from 'react';

function Counter() {
  // 2. Declare a state variable 'count' with initial value 0.
  // useState returns an array with two things:
  // [0] The current state value -> 'count'
  // [1] A function to update that state -> 'setCount'
  const [count, setCount] = useState(0);

  // 3. Create functions to update the state
  const increment = () => {
```

```
    setCount(count + 1); // Calls setCount to update 'count'
  };
  const decrement = () => {
    setCount(count - 1);
  };

  // 4. Use the state value in the JSX
  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
    </div>
  );
}
```

Important: Never modify state directly (e.g., `count = 1`). Always use the state setter function (`setCount`). This tells React that state has changed and a re-render is needed.

8. Handling Events

React events are similar to DOM events but use camelCase (e.g., `onClick` instead of `onclick`).

```
function Button() {
  const handleClick = (e) => {
    // 'e' is the React synthetic event object
    e.preventDefault();
    console.log('The button was clicked!');
  };

  return (
    <button onClick={handleClick}>
      Click Me
    </button>
  );
}
```

You can pass arguments to event handlers:

```
function ShoppingList() {
  const buyItem = (itemName) => {
    alert(`Buying ${itemName}`);
  };

  return (
    <ul>
```

```
    <li>Milk <button onClick={() => buyItem('Milk')}>Buy</button></li>
    <li>Bread <button onClick={() => buyItem('Bread')}>Buy</button></li>
  </ul>
 );
}
```

9. Conditional Rendering

You can render different JSX based on conditions. Use regular JavaScript operators like `if`, `&&`, or the ternary operator `? :`.

```
function UserGreeting({ isLoggedIn, username }) {
 // Method 1: if/else statement
 // if (isLoggedIn) {
 //   return <h1>Welcome back, {username}!</h1>;
 // } else {
 //   return <h1>Please sign up.</h1>;
 // }

 // Method 2: Ternary operator (inside JSX)
 return (
  <div>
   {isLoggedIn ? <h1>Welcome back, {username}!</h1> : <h1>Please sign up.</h1>}
  </div>
 );

 // Method 3: Logical && operator (for showing/hiding)
 return (
  <div>
   {isLoggedIn && <h1>Welcome back, {username}!</h1>}
   {/* Shows nothing if isLoggedIn is false */}
  </div>
 );
}
```

10. Lists and Keys

You can render lists of components using the array `map()` function.

```
function TodoList() {
 const todos = [
  { id: 1, text: 'Learn React', done: true },
  { id: 2, text: 'Build a project', done: false },
  { id: 3, text: 'Get a job', done: false }
```

```
  ];

  return (
    <ul>
      {todos.map((todoItem) => (
        // You must provide a unique 'key' prop for each list item.
        // It helps React identify which items have changed, been added, or removed.
        // Use a stable ID from your data, not the array index.
        <li key={todoItem.id}>
          {todoItem.text} - {todoItem.done ? '✅' : '❌'}
        </li>
      ))}
    </ul>
  );
}
```

The **Golden Rule of Keys:** Keys must be unique among siblings. They are not passed as props to the component; they are for React's internal use only.

Part 2: Core Concepts (Intermediate)

11. Component Lifecycle (and the `useEffect` Hook)

In class components, components go through a lifecycle: mounting (being added to the DOM), updating (when props or state change), and unmounting (being removed from the DOM). Methods like `componentDidMount` and `componentWillUnmount` let you run code at these specific times.

In functional components, the useEffect Hook is our one-stop-shop for all lifecycle-related side effects. A "side effect" is anything that affects something outside the scope of the function, like data fetching, setting up a subscription, or manually changing the DOM.

The **Basics** of `useEffect`:

```
import { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  // useEffect takes two arguments:
  // 1. A function containing the side effect code.
  // 2. An optional array of dependencies.
  useEffect(() => {
    // This function runs after every render by default.

    console.log('Effect running for userId:', userId);

    // Fetch user data from an API
    fetch(`https://api.example.com/users/${userId}`)
      .then(response => response.json())
      .then(data => {
        setUser(data);
        setIsLoading(false);
      });

  }, [userId]); // The dependency array. See rules below.

  if (isLoading) return <div>Loading...</div>;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
```

```
  );
}
```

Controlling When Effects Run with the Dependency Array:

- **No Dependency Array:** The effect runs after every render.

```
useEffect(() => { console.log('I run on every render!') });
// Use with caution! Can cause performance issues.
```

- **Empty Array `[]`:** The effect runs only **once, after the initial render**. This mimics `componentDidMount`.

```
useEffect(() => {
  console.log('I run only once, when the component mounts!');
  // Good for one-time setup (e.g., initial API call, adding event
listeners).
}, []);
```

- **Array with Values `[userId]`:** The effect runs after the initial render **and then only re-runs if any of those values have changed since the last render**. This mimics behavior from `componentDidUpdate`.

```
useEffect(() => {
  // This will run when the component first mounts AND whenever `userId`
changes.
}, [userId]);
```

Cleanup: Mimicking `componentWillUnmount`
Some effects need to clean up after themselves (e.g., canceling API requests, removing event listeners, clearing timers). To do this, return a cleanup function from your effect.

```
useEffect(() => {
  console.log('Setting up listener for window clicks');
  const handleClick = () => console.log('Window was clicked!');

  // Add event listener on mount
  window.addEventListener('click', handleClick);

  // Return a cleanup function
  return () => {
    console.log('Removing listener for window clicks');
    // Remove event listener on unmount (or before re-running the
effect)
    window.removeEventListener('click', handleClick);
  };
}, []); // Empty array means this setup/cleanup only happens on
```

```
mount/unmount
```

12. Hooks Deep Dive: `useState`, `useEffect`, Rules of Hooks

We've seen `useState` and `useEffect` in action. Now, let's solidify the rules.

The Two Golden Rules of Hooks:

1. **Only Call Hooks at the Top Level.** Don't call Hooks inside loops, conditions, or nested functions. This ensures Hooks are called in the same order each time a component renders, which is how React preserves state between multiple `useState` and `useEffect` calls.

```
// ❌ WRONG - Inside a condition
if (userIsLoggedIn) {
  const [data, setData] = useState(null);
}

// ✅ CORRECT - At the top level of the component
function MyComponent() {
  const [data, setData] = useState(null);
  if (userIsLoggedIn) {
    // use the state here
  }
}
```

2. **Only Call Hooks from React Functions.** Call them from within React functional components or from within custom Hooks (see later). Don't call them from regular JavaScript functions.

Functional Updates with `useState`:
When the new state depends on the previous state, pass a function to the state setter. This ensures you are working with the most up-to-date state value, especially important for multiple rapid updates.

```
function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    // ❌ Less reliable if multiple increments are queued
    // setCount(count + 1);

    // ✅ Functional update is always based on latest state
    setCount(prevCount => prevCount + 1);
  };

  const doubleIncrement = () => {
```

```
    increment(); // sets count to 0 + 1 = 1
    increment(); // sets count to 1 + 1 = 2 (works as expected)
    // Without functional updates, both would use the stale `count`
value of 0, resulting in 1.
  };

  return <button onClick={doubleIncrement}>Count: {count}</button>;
}
```

13. More Essential Hooks

`useContext`: Avoiding "Prop Drilling"

What if you need to pass a prop down through many levels of components? This is called "prop drilling" and it's tedious. The Context API provides a way to share data (a "theme", authenticated user, preferred language) through the component tree without passing props down manually at every level.

```
// 1. Create a Context
const ThemeContext = React.createContext('light'); // 'light' is the
default value.

// 2. Provide a context value high up in the tree
function App() {
  // The `value` prop here is what will be available to all consuming
descendants.
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

// A component in the middle doesn't have to pass the theme down.
function Toolbar() {
  return (
    <div>
      <ThemedButton />
    </div>
  );
}

// 3. Consume the context value in a deep child component
function ThemedButton() {
  const theme = useContext(ThemeContext); // 'dark'
  return <button className={theme}>I am styled by theme!</button>;
}
```

`useReducer`: A Powerful Alternative to `useState`

Ideal for managing complex state logic that involves multiple sub-values or when the next state depends heavily on the previous one. It's inspired by the Redux pattern.

```
import { useReducer } from 'react';

// 1. Define a reducer function: (state, action) => newState
// It takes the current state and an "action" (an object describing what
happened)
// and returns the new state.
function todoReducer(state, action) {
  switch (action.type) {
    case 'ADD_TODO':
      return [...state, { id: Date.now(), text: action.text, done: false
}];
    case 'TOGGLE_TODO':
      return state.map(todo =>
        todo.id === action.id ? { ...todo, done: !todo.done } : todo
      );
    case 'DELETE_TODO':
      return state.filter(todo => todo.id !== action.id);
    default:
      throw new Error('Unknown action type: ' + action.type);
  }
}

function TodoApp() {
  // 2. Initialize useReducer. It returns the current state and a
`dispatch` function.
  // The reducer function and initial state are passed as arguments.
  const [todos, dispatch] = useReducer(todoReducer, []);

  const handleAdd = (text) => {
    // 3. To update state, `dispatch` an "action"
    dispatch({ type: 'ADD_TODO', text });
  };

  return (
    <div>
      <button onClick={() => handleAdd('Learn useReducer')}>Add
Todo</button>
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            <span
              style={{ textDecoration: todo.done ? 'line-through' :
'none' }}
```

```
              onClick={() => dispatch({ type: 'TOGGLE_TODO', id: todo.id
})}
            >
              {todo.text}
            </span>
            <button onClick={() => dispatch({ type: 'DELETE_TODO', id:
todo.id })}>
              Delete
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

`useRef`: Accessing DOM Elements and Mutable Values

`useRef` returns a mutable "ref" object whose `.current` property is initialized to the passed argument. It has two main uses:

1. **Accessing DOM Elements:**

```
function TextInputWithFocusButton() {
  const inputEl = useRef(null); // Create a ref

  const onButtonClick = () => {
    // `current` points to the mounted text input element
    inputEl.current.focus();
  };

  return (
    <>
      {/* Attach the ref to a DOM element */}
      <input ref={inputEl} type="text" />
      <button onClick={onButtonClick}>Focus the input</button>
    </>
  );
}
```

2. **Keeping a Mutable Variable (that doesn't trigger re-renders):**

```
function Timer() {
  const [count, setCount] = useState(0);
  const intervalRef = useRef(); // Can hold a mutable value

  useEffect(() => {
    // Store the interval ID in the ref
```

```
    intervalRef.current = setInterval(() => {
      setCount(c => c + 1);
    }, 1000);

    // Clear on unmount
    return () => clearInterval(intervalRef.current);
  }, []);

  const stopTimer = () => {
    // Access the mutable value via .current
    clearInterval(intervalRef.current);
  };

  return (
    <>
      <div>Seconds: {count}</div>
      <button onClick={stopTimer}>Stop</button>
    </>
  );
}
// Changing intervalRef.current does NOT cause a re-render.
```

14. Forms in React

Controlled Components

The recommended way. Form data is handled by the React component's state. The input's value is controlled by React, and changes are handled by an `onChange` handler.

```
function ContactForm() {
  const [formData, setFormData] = useState({ name: '', email: '' });

  const handleChange = (event) => {
    const { name, value } = event.target;
    // Update state for the specific field that changed
    setFormData(prevData => ({
      ...prevData, // Copy the old state
      [name]: value // Update the specific field
    }));
  };

  const handleSubmit = (event) => {
    event.preventDefault(); // Prevent page refresh
    console.log('Form submitted with:', formData);
    // Send data to an API here
```

```
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Name:
        <input
          type="text"
          name="name"
          value={formData.name} // Controlled by state
          onChange={handleChange}
        />
      </label>
      <label>
        Email:
        <input
          type="email"
          name="email"
          value={formData.email} // Controlled by state
          onChange={handleChange}
        />
      </label>
      <button type="submit">Submit</button>
    </form>
  );
}
```

Uncontrolled Components

Form data is handled by the DOM itself. You use a `ref` to get the form values from the DOM when you need them. This is more like traditional HTML.

```
function ContactForm() {
  const nameInputRef = useRef(null);
  const emailInputRef = useRef(null);

  const handleSubmit = (event) => {
    event.preventDefault();
    // Get values directly from the DOM elements
    console.log('Name:', nameInputRef.current.value);
    console.log('Email:', emailInputRef.current.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Name:
```

```
        <input
          type="text"
          ref={nameInputRef} // Attach ref to access value later
        />
      </label>
      <label>
        Email:
        <input
          type="email"
          ref={emailInputRef}
        />
      </label>
      <button type="submit">Submit</button>
    </form>
  );
}
```

Use controlled components for most cases. They give you immediate validation, disable submit buttons conditionally, and enforce input formats.

15. Lifting State Up & Composition vs. Inheritance

Lifting State Up
When multiple components need to reflect the same changing data, lift the shared state up to their closest common ancestor. The parent component can manage the state and pass it down to the children via props.

- **Problem**: Two `TemperatureInput` components (one for Celsius, one for Fahrenheit) need to stay in sync.
- **Solution**: Lift the temperature value and the handler to their parent component (`Calculator`). The parent tells the children what to display via props.

Composition vs. Inheritance
React has a powerful composition model. The official advice is to **use composition instead of inheritance** to reuse code between components.

- **Containment**: Some components don't know their children ahead of time (like a generic `Box` or `Dialog`). Use the special `children` prop to pass children elements directly into their output.

```
function FancyBox({ children }) {
  return (
    <div className="fancy-box">
      {children} {/* The content passed between the tags appears here
*/}
```

```
    </div>
  );
}

function App() {
  return (
    <FancyBox>
      <h1>Welcome</h1>
      <p>This is all inside the box!</p>
    </FancyBox>
  );
}
```

- **Specialization**: Sometimes we think of components as being "special cases" of other components. This is also achieved by composition, where a more "specific" component renders a more "generic" one and configures it with props.

```
function Dialog({ title, message }) {
  return (
    <FancyBox> // Reusing the FancyBox component
      <h1>{title}</h1>
      <p>{message}</p>
    </FancyBox>
  );
}

function WelcomeDialog() {
  // A specialized Dialog
  return <Dialog title="Welcome" message="Thank you for visiting!" />;
}
```

16. Thinking in React: A Step-by-Step Guide to Building an App

This is the most important skill to learn. Here's the process:

1. **Break The UI Into A Component Hierarchy:** Draw boxes around every component and subcomponent. Name them. This is your component tree.
2. **Build A Static Version (without state):** Build components that only render based on props. This is a lot of typing without adding interactivity. Don't use state at all. **Top-down** (building parent components first) or bottom-up (building child components first) both work.
3. **Identify The Minimal (but complete) Representation Of UI State:** Ask three questions about each piece of data:
    a. Does it change over time? If not, it's not state.
    b. Is it passed in from a parent via props? If so, it's not state.
    c. Can you compute it based on existing state or props? If so, it's not state.
    d. The remaining data is your minimal state.

4. **Identify Where Your State Should Live: For each piece of state:**
    a. Identify every component that renders something based on that state.
    b. Find a common owner component (a single component above all others in the hierarchy that needs that state).
    c. Either the common owner or another component higher up should own the state.
    d. If you can't find a component where it makes sense to own the state, create a new component solely for holding the state and add it somewhere above the common owner component in the hierarchy.
5. **Add Inverse Data Flow (Pass Callback Handlers Down):** Since state is owned by a parent, child components need a way to update it. The parent passes down handler functions (e.g., `onAddTodo`) as props, which the children call when they need to update the state.

Part 3: Advanced Techniques & Ecosystem (Advanced)

17. Performance Optimization

React is fast by default, but in large apps, unnecessary re-renders can become a problem. A component re-renders if its state changes, its parent re-renders, or if the props it receives change.

`React.memo()`: Preventing Re-renders of Functional Components
`React.memo` is a Higher Order Component. It memoizes your component. If the props haven't changed, React will skip re-rendering the component and reuse the last rendered result.

```
import { memo } from 'react';

const ExpensiveComponent = memo(function ExpensiveComponent({ value }) {
  console.log('ExpensiveComponent rendered... it takes time!');
  // Very heavy computation based on value
  return <div>{value}</div>;
});

// Now, this component will only re-render if the `value` prop changes,
// even if its parent re-renders.
```

`useMemo`: Memoizing Expensive Calculations
Lets you cache (“memoize”) the result of a heavy calculation between re-renders.

```
function TodoList({ todos, filter }) {
  // This filtering function might be slow on a huge list...
  // const visibleTodos = todos.filter(todo =>
todo.text.includes(filter));

  // useMemo will only recalculate if `todos` or `filter` change.
  const visibleTodos = useMemo(() => {
    console.log('Recalculating filtered todos...');
    return todos.filter(todo => todo.text.includes(filter));
  }, [todos, filter]); // Re-run when these dependencies change

  return (
    <ul>
      {visibleTodos.map(todo => <li key={todo.id}>{todo.text}</li>)}
    </ul>
  );
}
```

`useCallback`: **Memoizing Functions**

Lets you cache a function definition between re-renders. This is useful when passing a callback to an optimized child component that relies on reference equality (like a component wrapped in `React.memo`) to prevent unnecessary re-renders.

```
function App() {
  const [todos, setTodos] = useState([]);
  const [filter, setFilter] = useState('');

  // ❌ This function is re-created on every render, changing its
reference.
  // const handleAddTodo = (text) => { setTodos([...todos, { text }]);
};

  // ✅ useCallback memoizes the function. It only changes if `setTodos`
changes (which it doesn't).
  const handleAddTodo = useCallback((text) => {
    setTodos(prevTodos => [...prevTodos, { text }]); // Using functional
update
  }, []); // `setTodos` is stable, so dependency array is empty.

  return (
    <div>
      <FilterInput value={filter} onChange={setFilter} />
      {/* AddTodo would re-render every time if handleAddTodo's
reference changed */}
      <AddTodo onAdd={handleAddTodo} />
      <TodoList todos={todos} filter={filter} />
    </div>
  );
}

const AddTodo = memo(({ onAdd }) => { // This component is memoized!
  console.log('AddTodo rendered');
  // ... some JSX with a form that calls onAdd
});
// Now, AddTodo won't re-render unnecessarily because `onAdd` prop's
reference stays the same.
```

Only use these optimizations when you have a measured performance problem. They add complexity.

18. Custom Hooks: Reusing Stateful Logic

Custom Hooks let you extract component logic into reusable functions. A custom Hook is a JavaScript function whose name starts with ”use” and that may call other Hooks.

Example: Custom `useLocalStorage` Hook

```
// useLocalStorage.js
import { useState, useEffect } from 'react';

function useLocalStorage(key, initialValue) {
  // 1. Get initial value from localStorage or use provided initialValue
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.log(error);
      return initialValue;
    }
  });

  // 2. Return a wrapped version of useState's setter function that...
  // ... persists the new value to localStorage.
  const setValue = (value) => {
    try {
      // Allow value to be a function so we have same API as useState
      const valueToStore = value instanceof Function ?
value(storedValue) : value;
      setStoredValue(valueToStore);
      // Save to local storage
      window.localStorage.setItem(key, JSON.stringify(valueToStore));
    } catch (error) {
      console.log(error);
    }
  };

  return [storedValue, setValue];
}

export default useLocalStorage;
```

Using the custom Hook in a component:

```
// Component.js
import useLocalStorage from './useLocalStorage';

function Component() {
```

```
  // Now we can reuse this state+persistence logic anywhere!
  const [name, setName] = useLocalStorage('name', 'Bob');
  const [age, setAge] = useLocalStorage('age', 30);

  return (
    <div>
      <input value={name} onChange={e => setName(e.target.value)} />
      <input
        type="number"
        value={age}
        onChange={e => setAge(Number(e.target.value))}
      />
    </div>
  );
}
```

Custom Hooks are a powerful mechanism for sharing logic without the complexity of render props or higher-order components.

19. Context API: Managing Global State

We saw a basic example earlier. Context is designed to share data that can be considered "global" for a tree of React components (e.g., current user, theme, preferred language). It's a great tool, but **don't reach for it to avoid passing props down a few levels** – that's what composition is for.

Advanced Pattern: Combining Context with `useReducer`
This is a powerful combination for managing more complex global state.

```
// contexts/AppContext.js
import { createContext, useReducer, useContext } from 'react';

// 1. Create a Context
const AppContext = createContext();

// 2. Define a reducer and initial state
const appReducer = (state, action) => {
  switch (action.type) {
    case 'SET_USER':
      return { ...state, user: action.payload };
    case 'SET_THEME':
      return { ...state, theme: action.payload };
    default:
      return state;
  }
};
```

```
const initialState = { user: null, theme: 'light' };

// 3. Create a Provider Component
export const AppProvider = ({ children }) => {
  const [state, dispatch] = useReducer(appReducer, initialState);

  // Value provided to consuming components: state and dispatch function
  return (
    <AppContext.Provider value={{ state, dispatch }}>
      {children}
    </AppContext.Provider>
  );
};

// 4. Create a custom hook for easy consumption
export const useAppContext = () => {
  const context = useContext(AppContext);
  if (!context) {
    throw new Error('useAppContext must be used within an AppProvider');
  }
  return context;
};


// index.js
import { AppProvider } from './contexts/AppContext';
// ...
root.render(
  <AppProvider>
    <App />
  </AppProvider>
);


// UserComponent.js
import { useAppContext } from './contexts/AppContext';

function UserComponent() {
  const { state, dispatch } = useAppContext();

  const login = () => {
    // Simulate API call, then dispatch an action to update global state
    dispatch({ type: 'SET_USER', payload: { name: 'Alice' } });
  };
```

```
  return (
    <div>
      <p>User: {state.user ? state.user.name : 'Guest'}</p>
      <button onClick={login}>Login</button>
    </div>
  );
}
```

20. Introduction to Routing with React Router

Single-Page Applications (SPAs) need a way to show different "pages" without refreshing the browser. React Router is the standard library for this.

Basic Setup:

```
npm install react-router-dom
```

Basic Usage:

```
// App.js
import { BrowserRouter as Router, Routes, Route, Link } from
'react-router-dom';
import Home from './Home';
import About from './About';
import UserProfile from './UserProfile';

function App() {
  return (
    <Router>
      <div>
        <nav>
          <Link to="/">Home</Link>
          <Link to="/about">About</Link>
        </nav>

        {/* <Routes> looks at its children <Route> elements and renders
the first one whose path matches the current URL. */}
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/users/:userId" element={<UserProfile />} /> {/*
Dynamic segment */}
          <Route path="*" element={<div>404 Not Found!</div>} /> {/*
Catch-all route */}
        </Routes>
```

```
      </div>
    </Router>
  );
}


// UserProfile.js
import { useParams } from 'react-router-dom';

function UserProfile() {
  // useParams hook gives you access to the dynamic parameters (:userId)
from the URL.
  let { userId } = useParams(); // If URL is /users/123, userId is "123"
  return <h2>User ID: {userId}</h2>;
}
```

21. Introduction to State Management: Redux Toolkit

For very large applications, managing all your state in React Context can become inefficient or hard to debug. **Redux** is a predictable state container. **Redux Toolkit (RTK)** is the modern, official, opinionated way to write Redux logic, drastically reducing the boilerplate.

Core Concepts:

- **Store:** A single source of truth for your application state.
- **Actions:** Plain objects that describe "what happened" (e.g., `{ type: 'todos/todoAdded', payload: 'Buy milk' }`).
- **Reducers:** Pure functions that determine how the state should change in response to an action. They take the current state and an action, and return the new state.
- **Slices:** RTK's `createSlice` function automatically generates action creators and action types based on reducer functions you provide, all bundled together.

Basic RTK Setup & Usage:

```
npm install @reduxjs/toolkit react-redux
```

```
// store.js
import { configureStore, createSlice } from '@reduxjs/toolkit';

// 1. Create a Slice
const todosSlice = createSlice({
  name: 'todos',
  initialState: [],
  reducers: {
    todoAdded: (state, action) => {
```

```
      // With RTK, we can write "mutating" logic thanks to Immer library
inside!
      state.push({ id: Date.now(), text: action.payload, done: false });
    },
    todoToggled: (state, action) => {
      const todo = state.find(todo => todo.id === action.payload);
      if (todo) todo.done = !todo.done;
    }
  }
});

// Extract the action creators
export const { todoAdded, todoToggled } = todosSlice.actions;

// 2. Create the Store
export const store = configureStore({
  reducer: {
    todos: todosSlice.reducer,
  },
});


// index.js
import { Provider } from 'react-redux';
import { store } from './store';
// ...
root.render(
  <Provider store={store}>
    <App />
  </Provider>
);


// TodoList.js
import { useSelector, useDispatch } from 'react-redux';
import { todoAdded, todoToggled } from './store';

function TodoList() {
  // 3. Read state from the store with `useSelector`
  const todos = useSelector(state => state.todos);
  // 4. Get the `dispatch` function to send actions
  const dispatch = useDispatch();

  const handleAdd = (text) => {
    dispatch(todoAdded(text)); // Dispatch the auto-generated action
creator
```

```
  };

  return (
    <div>
      {/* ... */}
      {todos.map(todo => (
        <div key={todo.id} onClick={() =>
dispatch(todoToggled(todo.id))}>
          {todo.text} - {todo.done ? '✅' : '❌'}
        </div>
      ))}
    </div>
  );
}
```

22. Testing React Apps

Testing is crucial for stable applications. The recommended tools are **Jest** (a test runner and assertion library) and **React Testing Library** (RTL, a library for testing React components in a way that resembles how users interact with them).

Writing a Simple Test:

```
// Greeting.js
function Greeting({ name }) {
  return <h1>Hello, {name}</h1>;
}

// Greeting.test.js
import { render, screen } from '@testing-library/react';
import '@testing-library/jest-dom'; // For extended matchers like
.toBeInTheDocument()
import Greeting from './Greeting';

test('renders greeting with a name', () => {
  // 1. Render the component with a prop
  render(<Greeting name="World" />);

  // 2. Query the rendered DOM for an element (getByText finds by text
content)
  const headingElement = screen.getByText(/hello, world!/i); //
Case-insensitive regex

  // 3. Make an assertion
  expect(headingElement).toBeInTheDocument();
```

```
});
```

RTL encourages testing behavior, not implementation. You don't test that a state variable was set; you test that the correct text appears on the screen after a user clicks a button.

23. What's Next?

- **Next.js**: The leading **React framework**. It enables features like Server-Side Rendering (SSR), Static Site Generation (SSG), and easy API routes. It simplifies routing, code splitting, and deployment. Essential for production-grade apps.
- **Gatsby**: A React-based framework optimized for building incredibly fast **websites and blogs** (content-driven sites). It pulls data from CMSs, Markdown, etc., at build time.
- **React Native**: Learn once, write anywhere. Build native mobile apps for iOS and Android using React and JavaScript.
- **TypeScript**: A superset of JavaScript that adds static types. Catching bugs at compile time rather than runtime is a game-changer for large React applications. Highly recommended.

Conclusion

This journey from console.log('Hello, World!') to discussing state management and testing frameworks covers the vast landscape of modern React development. Remember, the key to mastery is consistent practice. Build projects. Start small, then add complexity. Break things, then fix them. Read the official React docs – they are exceptional. Engage with the community.

React is a powerful tool that has fundamentally changed front-end development. Enjoy the process of learning it!

Happy Coding!